

Lab exercises

Track 16.4 –  LangGraph · 16-ai-tech-stack/tracks/langgraph/lab/EXERCISES.md

Lab — LangGraph

Work top to bottom. Each exercise states a **deliverable**; if you can't produce it, you haven't finished. Everything runs offline — no API key, no network. Keep using the deterministic fake models from `../code/` so your results are reproducible and your failures are your own.

```
/Users/srinip/ai-all/.venv/bin/pip install -r ../code/requirements.txt
```

1. Warm-up: break the state schema on purpose (45 min)

Open `../code/graph_basics.py`.

- **1a.** Delete the `Annotated[..., operator.add]` from `critiques` (make it plain `list[str]`). Re-run. How many critiques survive, and why is there no error?
- **1b.** Set `MAX_REVISIONS = 99` and `GOOD_ENOUGH = 0.99`. What stops the graph, after how many supersteps, and what did that cost you in "model calls"?
- **1c.** In `build_fanout_graph`, give `winner` the reducer `lambda old, new: max(old, new)`. Re-run section 4 with `safe=False`. Which branch wins, and is that deterministic? Justify your answer using supersteps, not intuition.

Deliverable: three sentences, one per part. Part 1c must name the mechanism.

2. Add a node that can go wrong (1h)

In `graph_basics.py`, add a `fact_check` node between `critique` and the router. It should raise `TimeoutError` on its **first two** invocations and succeed afterwards (a module-level counter is fine — this is the point of the exercise).

- **2a.** Run it with no retry policy. Record the exception and where the graph stopped.
- **2b.** Add `retry_policy=RetryPolicy(max_attempts=3, initial_interval=0.1, jitter=False)` to `add_node`. Re-run.
- **2c.** Now make it fail *permanently* instead. Add an `error_handler=` node (or a `try/except` inside the node that writes `{"fact_check": "unavailable"}`) so the graph degrades rather than dies.

Checks: - 2b completes; 2a does not. - You can state in one sentence why a retry policy would **not** have helped the `lookup_order` failure in `react_agent_from_scratch.py`.

3. Map-reduce with Send (1h)

Build a new graph: given a list of 7 short "documents", fan out one `score` task per document, then join into a `rank` node that returns the top 3.

Checks: - The fan-out width is read from state at runtime, not hard-coded. - The worker node uses a **different** state schema from the parent (`input_schema=`). - `stream_mode="updates"` shows all 7 workers completing in the **same superstep**. - Add an 8th document while the graph is compiled — no recompilation needed. Confirm it.

Deliverable: the graph file plus the streamed output showing one superstep of width 8.

4. An approval gate that a regulator would accept (1.5h)

Extend `../code/react_agent_from_scratch.py` with an `approval_gate` node between `agent` and `tools`, and a `SqliteSaver`.

- **4a.** Gate only *dangerous* tools. Add an `issue_refund` tool; `lookup_order` and `tax_rate` should run unattended. (Hint: the router decides, not the gate.)
- **4b.** The interrupt payload must contain enough for a human to decide without reading your code: tool name, arguments, the agent's stated reason, and the thread id.
- **4c.** Support all three resume values: `"approve"`, `"reject"`, and an **edit** that changes the tool arguments before execution.
- **4d.** Kill the process after the interrupt (literally `sys.exit()`), then resume in a second script run against the same SQLite file.

Checks: - A rejected refund produces a `ToolMessage` the agent can see and respond to — not a crash. - An edited refund executes with the human's arguments, and `get_state_history()` shows both the proposed and the executed values. - Your second process resumes without re-running any tool.

Deliverable: two scripts (`start.py`, `resume.py`) sharing one `.sqlite` file, plus the printed audit trail.

5. The idempotency bug, found and fixed (1h)

Take your section-4 graph. Put a side effect — append to a file, increment a counter — **above** the `interrupt()` call.

- **5a.** Demonstrate the double-fire. Show the count.
- **5b.** Fix it three different ways: (i) move it below the interrupt, (ii) move it to its own node, (iii) make it idempotent with a key derived from the checkpoint.
- **5c.** Which fix survives a *crash* mid-node, not just a resume? Explain why.

Deliverable: a 3-row table (fix, survives resume?, survives crash?) and one paragraph on 5c.

6. Time travel as an incident-review tool (1h)

Using `../code/checkpoint_and_interrupt.py` as a reference:

- **6a.** Run a thread to completion. Print `get_state_history()` with `checkpoint_id`, `next`, and one interesting state value per row.
- **6b.** Replay from the pending-approval checkpoint with a *different* resume value and observe that you get the **recorded** answer back. Explain why, in terms of crash recovery.
- **6c.** Fork properly (`update_state(cfg, None)`) and get a genuinely different outcome.
- **6d.** Use `update_state(..., as_node="X")` to rewrite one state value as if node X had written it, and show that the *edges out of X* fire next.

Check: you can state the difference between *replay* and *fork* in one sentence, and say which one your incident review actually needs.

7. Choose the architecture, and defend it (1h)

For each scenario below, pick **one** of: plain `while` loop · `create_agent` · `prebuilt` · hand-built `StateGraph` · supervisor multi-agent · a general workflow engine (Temporal et al.). Write two sentences: the choice, and the single fact that decided it.

1. A CLI tool that answers questions over a local codebase. Runs for 20 seconds, no persistence.
2. An insurance claims triage flow: extract → classify → **adjuster approves** → notify. SLA of one business day for the approval step.
3. A research assistant that must search 30 sources and synthesise. Cost is a concern.
4. A nightly job that reconciles two ledgers, calling an LLM only to explain the diffs.
5. A customer support bot with three escalation tiers and a strict "never issue a refund without a human" rule.

Check: at least one of your answers is "plain loop" and at least one is "not LangGraph". If not, you are choosing by fashion.

8. Capstone: the full graph (2h)

Build one offline graph demonstrating all of it, and hand it to someone else:

1. A typed state schema with at least three channels and **two different** reducers.
2. A cycle with a semantic exit *and* a budget in state.
3. A `Send`-based parallel fan-out somewhere in the middle.
4. A tool that fails and an agent that recovers from the failure.
5. An `interrupt()` approval gate backed by `SqliteSaver`.
6. A `stream_mode=["updates", "custom"]` narration that a UI could render directly.

7. A subgraph with a **different** state schema from its parent.
8. A short `README` section: "what this would cost as a plain `while` loop".

Check: a colleague clones it, runs `python capstone.py`, answers one approval prompt, and can explain from the output alone where the loop terminated and why. If they have to read your source to follow the run, your streaming is wrong.

9. Stretch: write the engine (2h)

Implement a ~120-line Pregel-ish engine yourself: a `Graph` class with `add_node`, `add_edge`, `add_conditional_edges`, channel reducers, supersteps, a dict-of-dicts checkpointer keyed by `(thread_id, step)`, and a `Pause` exception for interrupts.

Checks: - Your engine runs `graph_basics.py`'s writer graph unchanged, except for imports. - You can resume a paused thread from a checkpoint. - You can name **three** things `LangGraph` does that you skipped, and say what each is worth.

This is the exercise that kills framework mystique. The core ideas are small; the value is in the hundred edge cases you just decided not to handle.